

ZYProject 网络请求实现

ZYProject 项目网络请求实现详解

一、网络请求的整体架构

项目通过 **ZYNetworkRequest 单例类** 统一管理所有网络请求，核心基于第三方库封装实现，具体依赖如下：

- 基础请求框架：基于 **BANetManager** 封装，覆盖大部分常规请求；
- 补充请求方式：使用 **AFNetworking** 处理特殊场景（如图片上传）；
- 整体设计：通过单例模式确保请求配置统一，避免重复创建实例，便于维护与扩展。

二、核心网络请求方法

项目提供多种网络请求方法，适配不同业务场景（如普通表单提交、JSON 格式请求、图片上传），具体分类如下：

1. POST 请求（带提示 / 不带提示）

提供两种 POST 请求方法，核心区别在于“请求成功后是否显示‘成功！’提示”，满足不同交互需求：

方法名	核心特点	适用场景
<code>post:</code>	请求成功后显示“成功！”提示（使用 EasyTextView ）	需告知用户操作结果的场景（如提交表单）
<code>postNoPrompt:</code>	仅返回结果，不显示成功提示	无需用户感知的后台请求（如数据同步）

实现逻辑（以 `post:` 为例）

- 创建 **BADataEntity** 对象，配置请求 URL、参数、超时时间（统一 15 秒）；

2. 调用 `BANetManager` 的 `POST` 方法发起请求，通过 `SVProgressHUD` 显示加载状态；
3. 请求成功：解析 JSON 数据为 `ZYNetworkModel`，显示 “成功！” 提示，回调返回结果；
4. 请求失败：调用 `isNetWorking:` 方法转换错误信息，显示错误提示，回调返回错误。

2. Bean 类型的 POST 请求（JSON 格式）

针对 “请求体为 JSON 格式” 的场景，提供 `postBean:` 和 `postBeanNoPrompt:` 方法，与普通 `POST` 区别如下：

- 请求体格式：以 JSON 字符串作为请求体，而非普通表单参数；
- 底层框架：不依赖 `BANetManager`，直接使用 `NSURLConnection` 实现；
- 提示逻辑：与普通 `POST` 一致（`postBean:` 带成功提示，`postBeanNoPrompt:` 不带）。

3. 图片上传请求

提供两种图片上传方法，适配不同上传需求，底层依赖不同框架：

(1) `uploadImageWithOperations:`（基于 `AFHTTPSessionManager-`）

- 核心特点：支持多图上传，可配置图片压缩质量、参数名，适合复杂上传场景；
- 实现逻辑：
 1. 创建 `AFHTTPSessionManager` 实例，设置请求头与超时时间；
 2. 循环添加图片数据到请求体，指定图片参数名（如 `image`）与文件名；
 3. 发起 `POST` 上传请求，通过 `SVProgressHUD` 显示上传进度；
 4. 上传完成后，解析结果并回调（成功 / 失败逻辑与普通请求一致）。

(2) `uploadImage:`（基于 `BANetManager`）

- 核心特点：简化版图片上传，适合单图或简单上传场景，复用 `BANetManager` 的基础配置；
- 实现逻辑：将图片数据封装为 `BADataEntity` 参数，调用 `BANetManager` 的上传方法，流程与普通 `POST` 类似。

三、网络错误处理

项目通过 `isNetWorking:` 方法 统一处理网络错误，将系统错误码转换为用户友好的中文提示，核心逻辑如下：

1. 接收系统错误（如 `NSError`），提取错误码（`code`）；
2. 根据错误码分类处理：
 - 无网络（如 `NSURLErrorNotConnectedToInternet`）：提示 “请检查网络连接”；
 - 请求超时（如 `NSURLErrorTimedOut`）：提示 “请求超时，请稍后重试”；
 - 服务器错误（如 500 系列状态码）：提示 “服务器繁忙，请稍后再试”；
 - 其他错误：默认提示 “操作失败，请重试”；
3. 返回处理后的错误信息，用于界面提示（如 `EasyTextView` 显示）。

四、JSON 解析

项目使用 **YYModel 库** 实现 JSON 数据与模型对象的转换，性能高效且使用简洁，核心流程如下：

1. 网络请求成功后，获取服务器返回的 JSON 数据（`NSData` 格式）；
2. 将 JSON 数据转换为字典（`NSDictionary`），传入 `ZYNetworkModel` 的 `modelWithDictionary:` 方法；
3. 通过 `YYModel` 自动映射：将字典中的键值对匹配到 `ZYNetworkModel` 的属性（如 `code` 状态码、`data-` 业务数据、`message` 提示信息）；
4. 回调返回 `ZYNetworkModel` 实例，上层业务根据 `code` 判断请求是否成功，再解析 `data` 中的具体业务数据。

五、网络请求的使用流程

以 “提交用户表单” 为例，展示完整使用步骤：

1. **创建请求实体**：初始化 `BADataEntity`，配置请求参数；

```
// 1. 配置请求参数
NSDictionary *params = @{@"username": @"test", @"password": @"123456"};
// 2. 创建 BADataEntity，设置 URL、参数、超时时间
BADataEntity *entity = [BADataEntity new];
```

```
entity.url = @"https://api.zyproject.com/user/login";
entity.params = params;
entity.timeOut = 15; // 统一超时时间
```

2. **发起网络请求**：调用 `ZYNetworkRequest` 的 `post:` 方法，传入实体与回调；

```
// 1. 获取单例实例
ZYNetworkRequest *netRequest = [ZYNetworkRequest sharedInstance];
// 2. 发起 POST 请求
[netRequest post:entity result:^(ZYNetWorkModel * _Nullable model, NSError * _Nullable error) {
    // 3. 处理回调结果
    if (error) {
        // 失败：显示错误提示
        NSLog(@"请求失败：%@", error.localizedDescription);
    } else {
        // 成功：解析业务数据
        if ([model.code isEqualToString:@"200"]) {
            NSDictionary *userData = model.data;
            // 处理用户数据（如保存登录状态）
        } else {
            // 业务错误：显示提示（如“用户名或密码错误”）
            NSLog(@"业务错误：%@", model.message);
        }
    }
}];
```

3. **处理请求结果**：在回调中根据 `ZYNetWorkModel` 的 `code` 与 `error` 判断状态，执行对应业务逻辑（如页面跳转、数据刷新）。

六、网络请求的特点

1. **统一管理**：所有请求通过 `ZYNetworkRequest` 单例发起，配置（超时、请求头）统一，便于维护；

2. **友好交互**：

- 加载状态：使用 `SVProgressHUD` 显示加载中，提升用户感知；
- 结果提示：成功用 `EasyTextView` 显示“成功！”，失败显示中文错误提示；

3. **多场景适配**：支持普通 POST/GET、JSON 格式请求、图片上传，覆盖大部分业务需求；

- 4. **稳定配置**：统一设置 15 秒超时时间，避免请求长时间阻塞；
- 5. **高效解析**：基于 YYModel 实现 JSON 与模型转换，简化数据处理逻辑。

七、代码优化建议

现有网络框架已满足基础需求，但存在可优化空间，提升稳定性与可维护性：

1. 重构重复代码：

- `post`：与 `postNoPrompt`、`postBean`：与 `postBeanNoPrompt`：逻辑高度重复，可提取公共方法（如 `basePost:showPrompt:result`），通过“是否显示提示”参数区分，减少冗余；

2. 统一请求框架：

- 目前混用 `BANetManager`、`NSURLConnection`、`AFHTTPSessionManager` 三种方式，建议统一为 `AFHTTPSessionManager`（功能更全面、维护更活跃，支持请求取消、进度监听等）；

3. 修复线程问题：

- `postBean`：方法中，`SVProgressHUD` 的显示 / 隐藏操作在异步线程执行，可能导致界面卡顿或崩溃，需切换到主线程（`dispatch_async(dispatch_get_main_queue(), ^{ ... })`）；

4. 添加请求取消机制：

- 新增 `cancelRequestWithURL`：方法，支持根据 URL 取消指定请求，避免列表刷新、页面销毁时的无用请求浪费资源；

5. 补充参数验证：

- 在发起请求前，添加参数有效性验证（如必填参数非空、URL 格式正确），提前拦截无效请求，减少错误回调。

总结

ZYProject 的网络请求框架通过单例封装实现了“统一管理、多场景适配、友好交互”，简化了上层业务的请求逻辑。虽存在部分可优化点，但整体设计清晰，可维护性与复用性较高，能稳定支撑项目的网络需求。

（注：文档部分内容可能由 AI 生成）